“STUDENT ASSIGNMENT SIMILARITY ANALYZER”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

i

n

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

b

y

Praneeth Kumar Gadila
:2520030257

Guru Kiran
:2520030096

Kunal Shahane
:2520030275

Team Number:

Section:

Under the Guidance
of

Dr. S. Madhavi

A

s

s
o
c
i
a
t
e
P
r
o
f
e
s
s
o
r

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Founda
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, T
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Title of the Project"** is a record of Bonafide work of **Praneeth Kumar Gadila – 2520030257, Guru Kiran – 2520030096, Kunal Shahane – 2520030275** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Praneeth Kumar Gadila – 2520030257
Guru Kiran – 2520030096

Kone
(Deemed
Off-Campu
Phone No

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIF ICATE

This is certify that the mini project based report entitled **"Title of the Project"** is a bonafide work done and submitted by **Praneeth Kumar Gadila – 2520030257, Guru Kiran – 2520030096, Kunal Shahane – 2520030275** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide **Signature of the HOD**

(Dr. S. Madhavi)

Signature of the External Examiner

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. S. Madhavi, Associate Professor, Department of Computer Science and Engineering, for his guidance and support in preparing this project report. His academic guidance helped shape the presentation of the project and its connection to the concepts covered in Data Structures and Algorithms – 3.

I also extend my thanks to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic environment and encouragement required to undertake this work.

Finally, I thank everyone who contributed suggestions and support during the planning, implementation, and documentation of the Student Assignment Similarity Analyzer.

Praneeth Kumar Gadila – 2520030257

Guru Kiran – 2520030096

Kunal Shahane – 2520030275

Abstract

The Student Assignment Similarity Analyzer is a Java-based desktop application developed to compare student assignments and identify textual similarities between documents. The application accepts text files (TXT) and PDF files, extracts their contents, preprocesses the text, and calculates similarity scores using Term Frequency–Inverse Document Frequency (TF-IDF) and cosine similarity.

The project incorporates multiple Data Structures and Algorithms (DSA) techniques, including string matching, text indexing, dynamic programming, graph algorithms, network flow, approximation algorithms, randomized algorithms, and parallel computing concepts.

The primary objective is to demonstrate how theoretical DSA concepts can be applied to a practical problem involving document comparison, text analysis, and algorithmic efficiency.

INTRODUCTION

1.1 Overview

Students often submit assignments that cover similar topics, and reviewing many documents manually can take considerable time. The Student Assignment Similarity Analyzer automates part of this review by extracting text from assignment files, normalizing the content, representing terms numerically, and calculating similarity between document vectors.
The core comparison workflow uses TF-IDF to give terms weights based on their importance within a document collection and cosine similarity to compare the resulting vectors. A Java Swing interface provides access to document selection, comparison results, algorithm references or demonstrations, and history-related functionality. The project also serves as a learning platform for exploring different algorithmic approaches to text processing and computational problems.

1.2 Problem Statement

Manual identification of similar assignments becomes difficult as the number and length of documents increase. Basic character-by-character comparisons can repeat work, while documents with different wording may be difficult to compare using exact matching alone. A system is required to process TXT and PDF documents, extract and preprocess their text, calculate textual similarity, classify the result, and present the output in an understandable form.

1.3 Scope of the Project

The project focuses on textual similarity analysis for TXT and PDF assignments and on demonstrating DSA concepts through Java implementations. It includes document extraction,

preprocessing, TF-IDF vectorization, cosine similarity, threshold-based classification, and analysis-history functionality. The algorithm library contains examples of string matching, text indexing, dynamic programming, graph and network-flow algorithms, approximation methods, randomized techniques, hashing, and parallel-computing concepts.

The project is an educational prototype, not a complete institutional plagiarism-detection platform. It does not independently establish plagiarism or authorship and does not include a confirmed external-source plagiarism database. OCR support for scanned PDFs and full semantic understanding are not established as current features.

1.4 Objectives

Develop a Java desktop application for comparing student assignments.
Support TXT and PDF document inputs and extract their textual content.
Preprocess text to make document representations more consistent.
Calculate similarity using TF-IDF and cosine similarity.
Explore exact and approximate string-matching algorithms.
Demonstrate DSA concepts across Modules 1–6.
Maintain analysis history for reviewing previous comparisons.
Provide a user interface for accessing the application's functions.
Study algorithmic efficiency, computational complexity, and trade-offs.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

The system shall allow a user to select TXT and PDF assignment files.
The system shall read plain-text files and extract text from PDFs using Apache PDFBox.
The system shall preprocess extracted text before similarity analysis.
The system shall generate TF-IDF-based document vectors and calculate cosine similarity.
The system shall display a similarity score and a classification based on configured thresholds.
The system shall provide access to available algorithm references and demonstrations.
The system shall support saving and reviewing analysis history.
The system should handle invalid or empty inputs with appropriate feedback where implemented.

2.2 Non-Functional Requirements

Usability: the desktop interface should make the main comparison workflow accessible.
Modularity: GUI, extraction, preprocessing, similarity calculation, algorithms, and storage should be separated into components.
Maintainability: Java source files and the sources.txt compilation manifest should support organized builds.

Performance: document processing and algorithm choices should account for document length and collection size.
Reliability: document handling, similarity calculations, and history workflows should be validated with tests.
Portability: Java 17 provides a cross-platform runtime, subject to the installed JDK and dependency setup.
Interpretability: results should be described as textual similarity rather than conclusive proof of plagiarism.
Safety during testing: destructive test cleanup must run in an isolated temporary working directory.

3. SYSTEM DESIGN

3.1 High-level Architecture Diagram

The following diagram summarizes the main document-comparison path and the separate algorithm-reference path.

Component	Responsibility
User	Selects files and views results
Java Swing GUI	Presents controls, results, and history functions
TXT / PDF Input	Receives the selected documents
Text Extraction	Reads TXT content or extracts PDF text
Text Preprocessing	Normalizes and prepares extracted text
TF-IDF Vectorization	Converts terms into weighted numerical vectors
Cosine Similarity	Compares the document vectors
Score and Classification	Displays similarity according to configured thresholds
History and Storage	Retains analysis records for later review
Algorithm Reference / Demos	Provides access to the DSA algorithm library

Logical flow: User → Java Swing GUI → TXT/PDF Input → Text Extraction → Text Preprocessing → TF-IDF Vectorization → Cosine Similarity → Score and Classification → History and Storage.
The GUI also connects to the Algorithm Reference / Demos component, which provides access to DSA algorithm implementations. The algorithm library is not the same as the main TF-IDF/cosine-similarity pipeline.

3.2 Description of Each Layer

3.2.1 GUI Layer

The GUI layer is built with Java Swing. It provides the desktop window and access to document comparison, algorithm reference or demonstration functionality, and history-related features.

3.2.2 Document Input and Extraction Layer

This layer accepts TXT and PDF files. TXT content is read directly, while PDF text is extracted with Apache PDFBox. The extracted text is passed to preprocessing.

3.2.3 Text Preprocessing Layer

Preprocessing prepares the extracted content for comparison. Consistent tokenization and normalization help reduce differences caused by text formatting. Exact preprocessing details depend on the implemented code.

3.2.4 Similarity Engine

The similarity engine coordinates TF-IDF vectorization and cosine similarity. TF-IDF weights terms according to their frequency and distribution in the document collection. Cosine similarity compares the orientation of the vectors.

3.2.5 Algorithm Library

The algorithm library contains implementations or demonstrations covering string matching, indexing, dynamic programming, graph algorithms, network flow, approximation, randomized algorithms, hashing, and parallel computing. Individual algorithms may be standalone demonstrations rather than part of the primary document-comparison workflow.

3.2.6 Storage and History Layer

This layer retains analysis records and supports history review. The supplied documentation identifies storage/history functionality but does not establish a remote database or external service.

4. IMPLEMENTATION

4.1 Module-wise Implementation

4.1.1 Document Upload and Extraction

Users select assignment documents in TXT or PDF format. Plain-text input is read through Java file I/O, while PDF content is extracted through PDFBox. The extracted text is passed to the next stage.

4.1.2 Text Preprocessing

Extracted content is prepared for comparison by applying consistent text processing. The goal is to create usable text representations for term weighting and similarity calculation.

4.1.3 TF-IDF Vectorization

TF-IDF combines term frequency (TF) with inverse document frequency (IDF). A common formulation is $TFIDF(t,d) = TF(t,d) \times IDF(t)$, where $IDF(t) = \log(N / df(t))$; N is the number of

documents and df(t) is the number of documents containing term t. Implementations may use a smoothed variant.

4.1.4 Cosine Similarity and Classification

For vectors A and B, cosine similarity is calculated as (A · B) / (||A|| ||B||). Higher values indicate more similar term-weight patterns. The application converts the calculation into a displayed score and classifies it using configured thresholds. These thresholds indicate textual similarity and do not prove plagiarism.

4.1.5 History Management

The application includes functionality for storing and reviewing analysis records. History allows a user to revisit previous comparisons; the precise persistence format should be verified against the current source code.

4.1.6 DSA Algorithm Implementations

Module	Algorithms / Concepts Represented	Purpose in the Project
Module 1: Foundations	Text analysis, TF-IDF, cosine similarity, indexing, complexity	Connects document comparison with algorithmic analysis.
Module 2: String matching and indexing	Naive matching, KMP, Z algorithm, Rabin–Karp with double hashing, Aho–Corasick, Trie/TextIndex, suffix array, Kasai LCP, Levenshtein and Damerau–Levenshtein distance	Shows exact matching, multi-pattern search, indexing, and approximate string comparison.
Module 3: Dynamic programming	Weighted edit distance, Needleman–Wunsch, Smith–Waterman, Matrix Chain Multiplication, Optimal BST, Held–Karp TSP, Hamiltonian Path bitmask DP, Tree DP, rerooting, SOS DP	Demonstrates overlapping subproblems and optimization.
Module 4: Graphs and network flow	Ford–Fulkerson, Edmonds–Karp, Dinic, minimum cut, bipartite matching, minimum-cost maximum flow	Demonstrates flow networks, cuts, and matching.

Module 5: Complexity and approximation	Bounded SAT/3-SAT evaluation, Maximum Clique, Maximum Independent Set, Minimum Vertex Cover, Subset Sum, Vertex Cover 2-approximation, metric TSP 2-approximation, P/NP/co-NP, reductions, FPT, kernelization, PTAS/FPTAS/APX concepts	Compares exact combinatorial search with approximation and complexity concepts.
Module 6: Randomized and parallel	Randomized three-way Quicksort, Reservoir Sampling, Miller–Rabin, affine universal hashing, separate chaining, prefix sum/parallel scan, parallel reduction, Java parallel sorting, work/span/speedup, Brent's theorem	Illustrates randomization, hashing, sampling, and parallel concepts.

Coverage note: the supplied project documentation describes the algorithms represented in the codebase, but GUI availability and test coverage vary. Some concepts are theoretical explanations rather than complete implementations.

4.2 Front-end Logic (UI Rendering and State Management)

The front end is a Java Swing desktop interface rather than a web-based front end. Swing components display controls, accept user selections, and show results. Event listeners coordinate actions such as selecting files, starting comparisons, opening algorithm references or demonstrations, and accessing history. The interface is expected to update the displayed result after processing; exact widget names and state variables should be confirmed in the source code.

4.3 API / Library Integration

The documented external library integration is Apache PDFBox 3.0.2 for PDF text extraction. Java file I/O handles text-file access and local application data. The supplied documentation does not establish integration with a remote REST API, cloud service, or external plagiarism database; therefore, these are not described as completed features.

4.4 Similarity Calculation and Classification

The comparison pipeline extracts text, preprocesses it, constructs TF-IDF vectors, computes cosine similarity, and maps the score to a configured classification. The displayed percentage follows the application's calculation and conversion rules. Different thresholds or preprocessing decisions may change the classification.

4.5 Build and Source Organization

The project uses Java 17 and a source manifest named sources.txt to compile Java source files. The custom validation harness is named TestRunner.java. Compilation and test commands should be executed using the project's documented dependency setup.

4.6 Testing Safety

The supplied documentation notes that TestRunner.java performs destructive cleanup involving submission test data. It must be run from an isolated temporary working directory with its own test-data folder, not from the real project directory containing actual submissions.

5. RESULTS

5.1 Application Workflow

The user selects the assignment documents for comparison.
The application reads TXT content or extracts PDF text.
The extracted content is preprocessed.
TF-IDF converts the documents into weighted vectors.
Cosine similarity is calculated between the vectors.
The application displays a score and threshold-based classification.
The analysis record can be stored for later review.

5.2 Testing and Validation

The project documentation reports that an earlier audit by Copilot found Java 17 compilation and the custom validation suite passing, including regression checks for TXT/PDF processing, similarity, and storage/history workflows. This is a reported baseline for the audited project state, not an independently repeated test in this report.
The validation areas identified in the project documentation are:
TXT text extraction and PDF text extraction.
Document similarity calculation and threshold-based classification.
Submission storage and history-related workflows.
Selected algorithm correctness checks.
Invalid-input and edge-case handling where individual tests exist.
The available information does not establish exhaustive correctness testing for every algorithm or complete coverage of every syllabus topic.

5.3 Complexity Analysis

Algorithm	Typical Time Complexity	Space / Notes
Naive pattern matching	O(nm)	O(1) auxiliary space in a basic implementation
KMP	O(n + m)	O(m) prefix table
Z algorithm	O(n + m)	O(n + m) for combined sequence

Rabin–Karp	Expected O(n + m); worst case O(nm)	Depends on hash representation
Aho–Corasick	O(n + z) after construction	Fixed-alphabet standard representation; depends on automaton size
Levenshtein distance	O(nm)	O(nm), or O(min(n,m)) for score-only rolling rows
Cosine similarity	O(d)	For two dense vectors of dimension d
Matrix Chain Multiplication	$O(n^3)$	$O(n^2)$
Held–Karp TSP	$O(n^2\ 2^n)$	$O(n\ 2^n)$
Edmonds–Karp	$O(VE^2)$	Depends on graph representation
Vertex Cover 2-approximation	O(V + E)	With standard edge-picking implementation
Randomized Quicksort	Expected O(n log n); worst $O(n^2)$	Recursion-stack space depends on partition balance

Here, n and m represent text and pattern lengths where applicable, V and E represent graph vertices and edges, d is the vector dimension, and z is the number of pattern occurrences reported. These are theoretical reference bounds, not measured execution times for this application.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

Processing both TXT and PDF documents requires separate input-handling paths.
Text extracted from documents can contain formatting differences that affect comparison.
Similarity scores can be misunderstood as proof of plagiarism if their meaning is not clearly explained.
The algorithm library spans several DSA areas, and not every algorithm has a dedicated interactive demonstration or direct automated test.
Some algorithms have high time or space costs for larger inputs, particularly exponential-time dynamic-programming and combinatorial-search methods.
The test harness has destructive cleanup behavior and must be isolated from real submission data.

6.2 Solutions Applied

Use Java file I/O for TXT files and Apache PDFBox for PDF extraction.
Apply a preprocessing stage before TF-IDF vectorization to improve consistency.
Use TF-IDF and cosine similarity as a defined, repeatable baseline for lexical comparison.

Separate the main similarity workflow from the broader algorithm library.
Use modular Java components and a sources.txt manifest to organize compilation.
Use a custom validation harness for core workflows, while running it in an isolated temporary working directory.

6.3 Limitations of the System

A similarity score does not independently prove plagiarism or identify the original author.
Results depend on extraction quality, preprocessing, vocabulary, and configured thresholds.
OCR for scanned PDFs is not established as a supported feature.
TF-IDF and cosine similarity measure lexical patterns and do not fully understand semantic meaning.
Paraphrased content may not receive a high similarity score.
Performance depends on document size, collection size, and algorithm choice.
Some algorithms are standalone demonstrations rather than parts of the primary comparison pipeline.
Not every syllabus topic has a dedicated GUI demonstration or exhaustive automated tests.
The project does not include a complete external-source plagiarism database.
Advanced structures such as suffix automata and suffix trees are not established as implemented; several concepts remain theoretical or partial.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

The Student Assignment Similarity Analyzer demonstrates how a practical text-analysis task can be approached using Java and established Data Structures and Algorithms techniques. It combines TXT/PDF text extraction, preprocessing, TF-IDF vectorization, cosine similarity, threshold-based classification, and analysis-history functionality.
The associated algorithm library represents concepts from string matching, text indexing, dynamic programming, graph algorithms, network flow, NP-completeness, approximation, randomization, hashing, and parallel computing. The project provides a foundation for studying algorithm selection, computational complexity, and modular software design. Further validation and integration work are required before treating all listed algorithmic topics as fully tested features.

7.2 Future Work

Add OCR support for scanned documents.
Improve text normalization and multilingual document handling.
Introduce semantic similarity methods to complement lexical similarity.
Generate detailed reports with highlighted matching passages.
Provide dedicated interactive demonstrations for each required syllabus topic.
Expand automated tests for algorithm correctness, edge cases, and invalid inputs.
Benchmark algorithms using different document sizes and input distributions.

Improve navigation, score explanation, and result visualization.
Allow configurable thresholds and more detailed analysis reports.
Extend graph-based demonstrations and algorithmic reductions according to the syllabus.
These items are proposed enhancements and should not be presented as completed functionality.

7.3 Skills Learned During Development

Java programming and desktop GUI development using Swing.
File handling and integration of a third-party PDF text-extraction library.
Text preprocessing and document representation using TF-IDF.
Similarity measurement using cosine similarity.
Understanding and comparing string-matching and text-indexing algorithms.
Applying dynamic programming, graph algorithms, network flow, approximation, and randomized techniques.
Reasoning about time complexity, space complexity, and algorithmic trade-offs.
Modular project organization, compilation, and validation.
Technical documentation, limitation analysis, and planning future improvements.
REFERENCES

Apache PDFBox, version 3.0.2, official project documentation: https://pdfbox.apache.org/
Java Platform, Standard Edition 17, official documentation: https://docs.oracle.com/en/java/javase/17/
Java Swing, official tutorial: https://docs.oracle.com/javase/tutorial/uiswing/
Cosine similarity and TF-IDF: standard information-retrieval and text-mining concepts; use the specific course text or lecture material prescribed by the instructor.
Algorithm complexity bounds: standard algorithm-analysis references and the DSA-3 course materials.
Note: Add any additional textbooks, lecture notes, websites, or code libraries actually consulted during implementation before final submission.